

Introduction

Unmanned vehicles, or UVs are one of the most promising technologies of today, and thus they are being actively developed. UVs provide several key advantages over human-driven vehicles, including higher road safety caused by eliminating human factors such as fatigue, inattention, or traffic violations, reduced operating costs for vehicles due to the absence of a need for a driver and more efficient fuel consumption and increased capacity of transportation infrastructure due to data exchange between UVs.

There are several approaches to organizing autopilot: classical (or modular) and end-to-end.

In the classical approach, the autopilot consists of several subsystems: mapping subsystem, sensor subsystem, perception subsystem (which uses sensor information to build a high-level picture of the world around the vehicle), localization subsystem (determining one's position in the environment), trajectory planning subsystem, prediction subsystem, control subsystem (converting the planned trajectory and feedback into signals for the actuator), actuators, simulation.

Figure 1 shows the interaction diagram of modules in the classical approach. Sensors extract information from the external world about external objects (cameras, lidars, radars) and the vehicle itself (GPS, IMU, odometer). Localization and perception subsystems make use of sensor information, as well as each other's results to perform their task. The mapping subsystem contains information about predominantly static surroundings, which can be used by the localization subsystem and updated by the perception subsystem. Data from the mapping, localization, and perception subsystems are used by the trajectory planning subsystem to create a trajectory in two stages: first an approximate global trajectory connecting the current position of the vehicle with the destination point, followed by a more precise local trajectory. To construct the local trajectory, data from the trajectory prediction subsystem is also used. The control subsystem drives the vehicle along the created local trajectory, sending appropriate signals to the actuators. Simulation subsystem is typically used instead of a real vehicle for testing purposes.

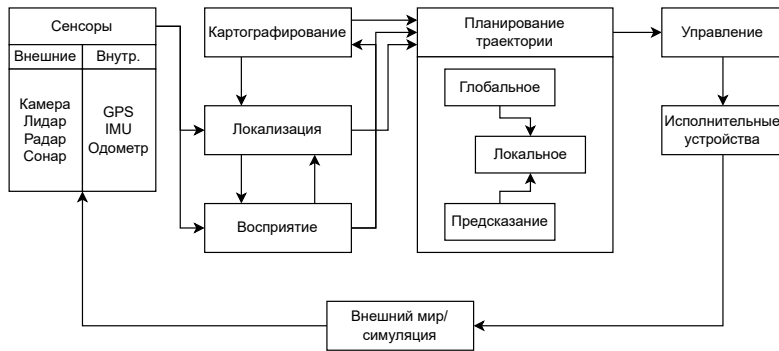


Figure 1 – Classical architecture diagram

In the end-to-end approach (shown on Figure 2), vehicle control is performed by a single model that receives signals from sensors and produces control signals for the vehicle. This approach is less modular and is more difficult to test compared to the classical approach. Additionally such a model is difficult to train due to its large size.

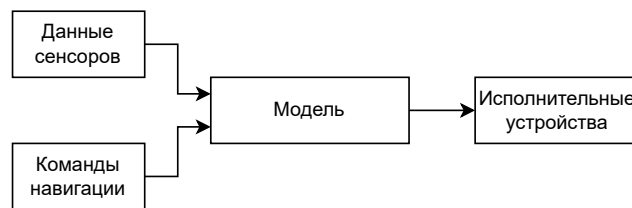


Figure 2 – End-to-end architecture diagram

The perception subsystem architecture depends on the sensor suite used, which are compared in table 1. Two approaches are the most common: a camera-only approach, and sensor fusion approach, which combines data from multiple sensor types, such as cameras, lidar and radar. The second approach provides higher accuracy, but requires significantly more expensive equipment compared to the first approach.

Table 1 – Comparison of several sensor types

Sensor type	Camera	Lidar	Radar
Angular resolution	High, 0.05° for a 2K camera with a 90° field of view	About 0.25° , reaching 0.1° [1]	Reaches 0.5° [2]
Distance resolution	Additional processing is needed to extract distance	High	Medium
Affected by weather	Yes	Yes	No
Affected by lighting	Yes	No	No
Interferes with other sensors	No	Yes	Yes, but there are compensation methods
Directly available information	Color/Texture	Distance	Speed, Distance

This work proposes creation of a perception subsystem that only uses data from stereo cameras.

1 Analysis of Methods and Algorithms for Video Data Processing by Computer Vision Systems of Unmanned Vehicles

1.1 Description of Existing Approaches to Building Video Data Analysis Architecture

Two approaches are distinguished for building the perception module architecture [3]: modular, where the perception module is divided into submodules, and end-to-end, where a single model performs the role of the perception module.

One example of a perception system using a modular approach is Apollo version 9 from Baidu. In this system, data processing (detection and tracking of objects) is performed separately for each type of sensor, after which data fusion is performed. On images, object detections are made by a model based on the YOLO architecture. On point clouds, object detections are created by the Center-Point model. Additionally, lane markings and traffic light state are extracted from camera data using darkSCNN and Faster-RCNN models respectively. The architecture diagram is shown in Figure 3.

An example of a system using an end-to-end approach is Tesla's autopilot. This system uses only camera data. An image from each camera passes through a chain of feature extraction consisting of an alignment layer that compensates for camera distortion, RegNet, and a bidirectional feature pyramid that connects features from several of the most recent RegNet layers. Features from each camera are combined using a transformer layer, after which they are stored in a feature queue consisting of two parts: a spatial queue used for predicting road geometry, and a temporal queue for predicting currently occluded object positions. The video module is a spatial recurrent neural network that combines features stored in the queue, after which modules are used to solve object detection, lane marking, and traffic light detection tasks. The architecture diagram is shown in Figure 4.

A modular system is simpler to train and reason about, but overall accuracy depends heavily on the ability of modules to resist errors. Meanwhile, the end-to-end approach provides an opportunity to train the entire model at once, making it more robust to such errors [3].



Figure 3 – Architecture diagram of the perception module of the Apollo autopilot system



Figure 4 – Architecture diagram of the perception module of the Tesla autopilot system

Video processing is a computationally intensive task that, in the case of autonomous driving systems, must be performed in real time at least 10 times per

second with low power consumption. Therefore, various accelerators are widely used when building hardware platforms: GPU, neural network accelerator (NNA), image signal processor (ISP), field-programmable gate array (FPGA), and others. Moreover, to increase performance, the ability to exchange data directly between accelerators without CPU involvement is added, as is done in the FSD system-on-chip used by Tesla to run its autopilot, whose diagram is shown in Figure 5.

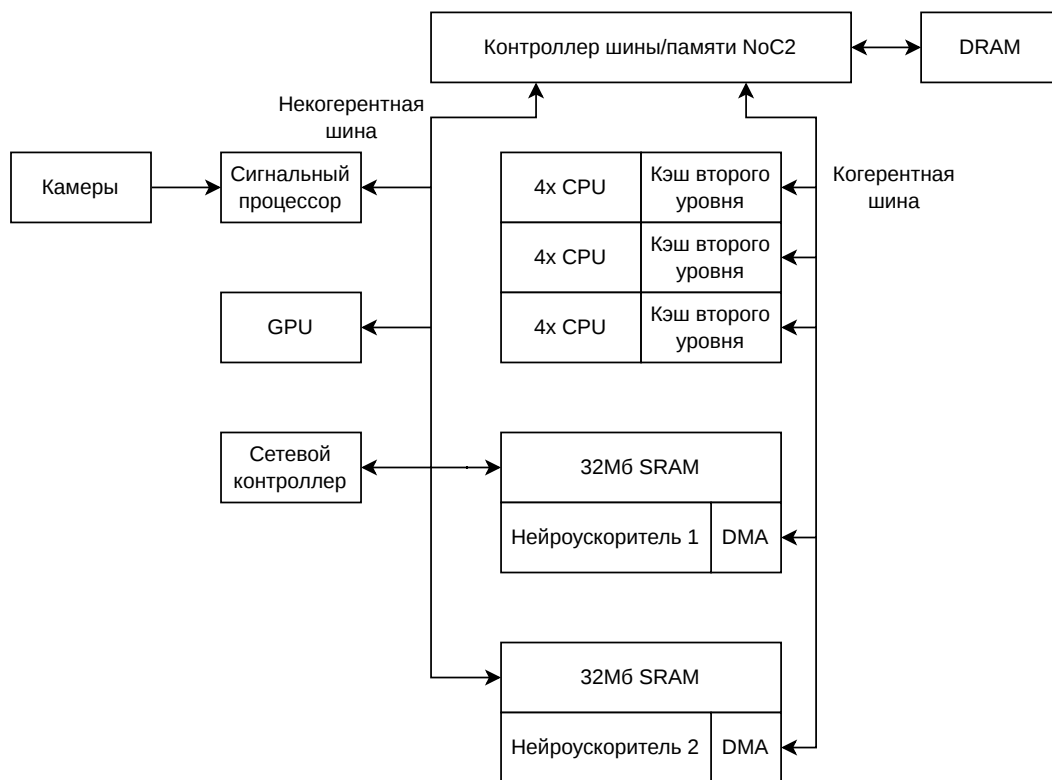


Figure 5 – Hardware architecture of Tesla FSD system-on-chip [4]

1.2 Stereo Camera Calibration

The process of camera calibration solves several tasks: for a single camera, it finds or refines the internal (a.k.a. intrinsic) parameters of the camera – those being focal lengths (f_x and f_y), principal points (c_x , c_y), and lens distortion parameters. For a pair of cameras combined into a stereo pair, in addition to calibrating each single camera, it finds or refines the external (a.k.a. extrinsic) parameters of the stereo camera – the offset vector and rotation from one camera to the second.

Calibration is done by taking photos of known calibration patterns are used for camera calibration, whose key points have a well-known position relative to each other and whose pixel coordinates can be accurately determined. Examples of such patterns include: a chessboard grid, symmetric and asymmetric grid of

circles, a pattern of triangles arranged in a specific manner with an AprilTag marker in the center, as used in [5] (shown in Figure 6).

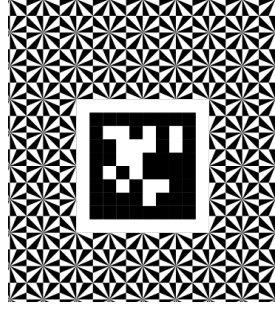


Figure 6 – Fragment of the pattern proposed in [5]

The camera model relates the coordinates of points on an image (in pixel coordinates) to the three-dimensional coordinate of a point in the captured scene. The basic model used is the pinhole camera model – a camera without lenses in which light passes through a hole of a negligibly small size. The model has the following form:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix},$$

where u, v are pixel coordinates of a point on the image, X_c, Y_c, Z_c are three-dimensional coordinates of a point in the captured scene, f_x, f_y are the focal lengths of the camera in pixels, for real cameras usually $f_x = f_y$, c_x, c_y are the principal points.

Real cameras have lenses that introduce distortions into images, so there are various extended camera models that account for these distortions. The OpenCV library uses a parametric camera model with the following form [6]:

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} f_x x'' + c_x \\ f_y y'' + c_y \end{bmatrix},$$

$$\begin{bmatrix} x'' \\ y'' \end{bmatrix} = \begin{bmatrix} x' \frac{1+k_1 r^2+k_2 r^4+k_3 r^6}{1+k_4 r^2+k_5 r^4+k_6 r^6} + 2p_1 x' y' + p_2 (r^2 + 2x'^2) + s_1 r^2 + s_2 r^4 \\ y' \frac{1+k_1 r^2+k_2 r^4+k_3 r^6}{1+k_4 r^2+k_5 r^4+k_6 r^6} + p_1 (r^2 + 2y'^2) + 2p_2 x' y' + s_1 r^2 + s_2 r^4 \end{bmatrix},$$

$$r^2 = x'^2 + y'^2,$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{X_c}{Z_c} \\ \frac{Y_c}{Z_c} \end{bmatrix},$$

where $Z_c \neq 0$, k_1 , k_2 , k_3 , k_4 , k_5 and k_6 are coefficients that model the imperfect shape of lenses and are used to compensate for radial distortion, p_1 and p_2 are coefficients that account for tangential distortion, modeling lens center displacement, and s_1 , s_2 , s_3 , s_4 are thin prism model coefficients, modeling lens tilt.

Parametric models have a limited number of parameters and account for only a portion of possible distortions. To account for arbitrary distortions, including the relevant case of compensating for distortions caused by the camera being located behind the windshield, generic models are used, such as the one proposed in [5]. Such models do not account for lens optics. Instead, they use a large number of parameters (on the order of tens of thousands). Thus, in [5], a grid of control points is created, each containing a normalized direction vector. Using cubic interpolation, a normalized vector $\overrightarrow{d(u, v)}$ is determined for each pixel of the image with coordinates $\begin{bmatrix} u \\ v \end{bmatrix}$. The model has the following form:

$$s \times \overrightarrow{d(u, v)} = \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}.$$

The disadvantages of the generic model compared to parametric models are higher implementation and calibration complexity, as larger number of parameters requires a larger number of calibration images. At the same time, the influence of errors from parametric models on the final depth is insignificant (on the order of several centimeters at distances of tens of meters), so the use of a generic model might not be justified.

For depth estimation algorithms to work, images in a stereo pair must be rectified, meaning one of the image axes (in this case u) must align with the direction from one camera to the other. Thus, corresponding pixels of the same object will have the same coordinate on one axis, while the second will differ by the amount of disparity. Formally, let $\begin{bmatrix} u_l \\ v_l \end{bmatrix}$ be the coordinates of a pixel on the image from the left camera, $\begin{bmatrix} u_r \\ v_r \end{bmatrix}$ be the coordinates of the corresponding pixel on the image from the right camera, d be the amount of mismatch caused by the parallax effect. If the u axis aligns, then $v_l = v_r$, $u_l + d = u_r$.

During calibration, the external parameters of the stereo camera are determined: R and T – the rotation matrix and offset vector from the first camera to the second.

Image rectification is performed together with distortion compensation as follows:

$$\text{rectified}(u, v) = \text{src}(\text{map}_x(u, v), \text{map}_y(u, v)),$$

where $\text{rectified}(u, v)$ is the rectified image, $\text{src}(u, v)$ is the original image from the camera, $\text{map}_x(u, v)$ and $\text{map}_y(u, v)$ are coordinate mapping maps of the rectified image and the original.

The mapping maps are constructed using the following formulas:

$$x = \frac{u - c'_x}{f'_x},$$

$$y = \frac{v - c'_y}{f'_y},$$

$$\begin{bmatrix} X \\ Y \\ W \end{bmatrix} = R_c^{-1} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

$$x' = \frac{X}{W},$$

$$y' = \frac{Y}{W},$$

$$r^2 = x'^2 + y'^2,$$

$$x'' = x' \frac{1 + k_1 r^2 + k_2 r^4 + k_3 r^6}{1 + k_4 r^2 + k_5 r^4 + k_6 r^6} + 2p_1 x' y' + p_2 (r^2 + 2x'^2) + s_1 r^2 + s_2 r^4,$$

$$y'' = y' \frac{1 + k_1 r^2 + k_2 r^4 + k_3 r^6}{1 + k_4 r^2 + k_5 r^4 + k_6 r^6} + p_1 (r^2 + 2y'^2) + 2p_2 x' y' + s_1 r^2 + s_2 r^4,$$

$$\text{map}_{x(u,v)} = x'' f_x + c_x,$$

$$\text{map}_{y(u,v)} = y'' f_y + c_y,$$

where k_1, k_2, k_3, k_4, k_5 and $k_6, p_1, p_2, s_1, s_2, s_3, s_4$ are the coefficients described above, R_c is the rectification matrix for this camera, f'_x, f'_y, c'_x, c'_y are the new focal lengths and principal points.

Additionally, one can compute the perspective matrix Q , used to transition from pixel coordinates and disparity to three-dimensional coordinates of a point:

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_{x_1} \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & -\frac{1}{T_x} & \frac{c_{x_1} - c_{x_2}}{T_x} \end{bmatrix}$$

1.3 Depth Estimation

There are three main methods for depth estimation using cameras: stereo pair-based using classical algorithms, stereo pair-based using deep learning, and single-image-based using deep learning.

In stereo pair-based depth estimation, the parallax effect is used: the apparent position of objects changes differently depending on the distance to them when the observer's position changes. A disparity map is used to describe the change in object position; each pixel of this map contains the distance from a pixel on the left image to its corresponding pixel on the right image. The disparity map can be converted into a point cloud in the world coordinate system by multiplying with the perspective matrix as follows:

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ w \end{bmatrix} = Q \times \begin{bmatrix} x \\ y \\ \text{disparity}(x, y) \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} \frac{x_1}{w} \\ \frac{y_1}{w} \\ \frac{z_1}{w} \end{bmatrix}$$

Where Q is a 4-by-4 perspective matrix, x, y are pixel coordinates, $\text{disparity}(x, y)$ is the value of the disparity map at the specified coordinates, x_w, y_w, z_w are the coordinates of the point in the world coordinate system.

Examples of classical algorithms include block matching and its improved version, semiglobal block matching [7].

Methods based on machine learning train a model to obtain a disparity map directly from a stereo pair. This method usually has higher accuracy compared to classical algorithms. Examples of such models are PSMNet [8], BANet [9].

The architecture of PSMNet consists of several steps: images from the right and left cameras are taken as input, each image is processed by a CNN for feature extraction and a Spatial Pyramid Pooling module, which allows taking into account information from neighboring parts of the image. The spatial pyramid pooling module diagram is shown in Figure 7.

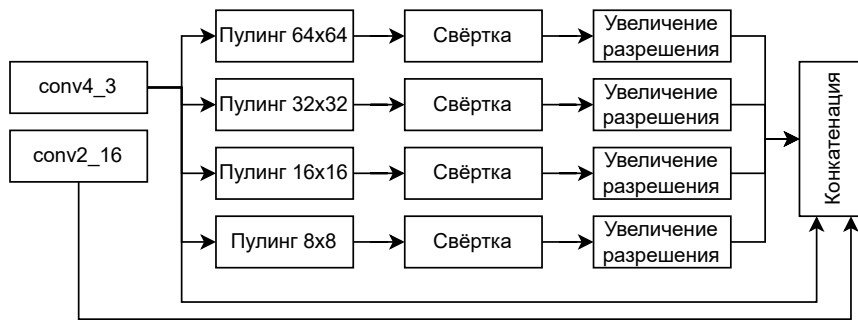


Figure 7 – Spatial Pyramid Pooling Module. Here conv4_3 and conv2_16 are outputs of CNN feature extraction layers

After the spatial pyramid pooling module, features from two images are combined into a cost volume. The cost volume is a four-dimensional tensor with dimensions (width, height, disparity, features) and is computed for each possible disparity as follows:

$$Cv(x, y, d, f) = L(x + d, y, f) * R(x, y, f),$$

where $Cv(x, y, d, f)$ is the cost volume, $L(x, y, f)$ and $R(x, y, f)$ are the features of the left and right images.

The next step is 3D convolution: 12 layers with 3-by-3-by-3 kernels, after which regression is performed to obtain a disparity map. The final disparity map $\hat{d}$ is computed as a weighted sum of disparity values by their probabilities:

$$\hat{d} = \sum_{d=0}^{D_{\max}} d \times \text{softmax}(-c_d)$$

The BANet architecture proposes a new approach – Bilateral Aggregation – for extracting a disparity map from the cost volume, which solves the disadvantages of using 3D convolution (high computational complexity) or 2D convolution

(low accuracy, blurry edges, loss of detail, errors in uniform parts of the image). The new approach is as follows: maps of detailed (A) and uniform ($A - 1$) parts of the image are created as follows:

$$S = \text{Concat}\left(\left[\text{Conv}\left(F_{1,16}^{\text{up}}\right), \text{Conv}\left(F_{1,8}^{\text{up}}\right), \text{Conv}\left(F_{1,4}\right)\right]\right),$$

$$A = \sigma(\text{Conv}(S)).$$

Here $F_{1,16}^{\text{up}}$, $F_{1,8}^{\text{up}}$, $F_{1,4}$ are features from the three last CNN feature extraction layers from the left image, upsampled to 1/4 of the input image resolution, $\text{Concat}()$ is the concatenation operation, $\text{Conv}()$ is the convolution operation, $\sigma()$ is a sigmoid serving as an activation function.

After that, the overall cost volume C_{cor} is split into two – C_d , which accounts for the detailed part of the image, and C_a , which accounts for the uniform part of the image:

$$C_d = A \odot C_{\text{cor}}$$

$$C_s = (1 - A) \odot C_{\text{cor}}$$

2D convolution is used to extract a disparity map:

$$C'_d = G_d(C_d)$$

$$C'_s = G_s(C_s)$$

The modules G_d and G_s have the same architecture but separate weights.

At the last step, two disparity maps are combined into one:

$$C_{\text{agg}} = A \odot C'_d + (1 - A) \odot C'_s$$

Methods that use only a single image estimate depth from various features, like illumination and texture. Such methods have significantly lower accuracy but are less demanding on hardware, requiring only one camera instead of two.

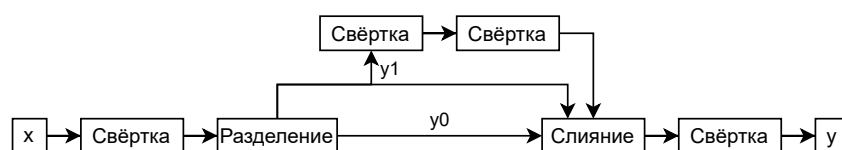
Thus, this work will use a machine learning-based method for extracting depth from stereo images due to higher accuracy, and will use the BANet architecture model, which provides high accuracy and high performance thanks to bilateral aggregation.

1.4 Semantic Segmentation

For automotive vision systems, the task of detecting and classifying objects in an image with pixel-level delineation of the area occupied by each object is relevant. This task is called semantic instance segmentation [10]

Semantic instance segmentation is only performed by using machine learning. The following model architectures will be considered below: YOLOv8-seg, RF-DETR-seg.

YOLOv8-seg is a convolutional neural network. The key feature of YOLOv8 compared to previous versions is the use of C2f layers [11]. C2f are a modified version of Cross Stage Partial Bottleneck. A convolution operation is performed on the input tensor, after which the tensor is split into two tensors of equal size, after which one half passes through a bottleneck represented by two convolution operations, after which all three parts are concatenated into one tensor, after which another convolution operation is performed. The layer diagram is shown in Figure 8.



RF-DETR-seg is an extended version of the RF-DETR (Roboflow detection transformer) detector model based on transformers, founded on the LW-DETR architecture, to which a Segmentation Head module was added. The key feature of this family of models is the use of Neural Architecture Search during training, allowing multiple model configurations to be trained simultaneously, for example, with different input image resolutions, number of decoder layers, and query tokens, while selecting the configuration with maximum accuracy for each performance level.

The RF-DETR-seg architecture is shown in Figure 9 and consists of a pre-trained ViT backbone (DINOv2-S or DINOv2-B) for feature extraction from the input image, serving as an encoder, a convolutional projector (C2f layer) that combines features from the three last layers of the encoder, a group of decoders consisting of up to 6 decoder layers, a detection head module, and a segmentation module.

During testing on the COCO dataset, the smallest RF-DETR-seg model (nano) has higher accuracy compared to YOLOv8-seg and YOLOv11-seg models

of sizes nano, small, and medium at similar execution times. The medium-sized RF-DETR-seg has similar accuracy to MaskDINO with significantly lower execution time [12]. Accuracy and execution time values are presented in table 2.

Table 2 – Comparison of instance segmentation models

Model	AP	AP50	Execution Time (ms)
YOLOv8-seg (nano)	28.3	45.6	3.5
YOLOv11-seg (nano)	30.0	47.8	3.6
YOLOv8-seg (small)	34.0	53.8	4.2
YOLOv11-seg (small)	35.0	55.4	4.6
YOLOv8-seg (medium)	37.3	58.2	7.0
YOLOv11-seg (medium)	38.5	60.0	6.9
FastInst (R50)	34.9	56.0	39.6
RF-DETR-seg (nano)	40.3	63.0	3.4
RF-DETR-seg (small)	43.1	66.2	4.4
RF-DETR-seg (medium)	45.3	68.4	5.9
MaskDINO (R50)	46.3	69.0	242
RF-DETR (large)	47.1	70.5	8.8

The selection of the RF-DETR-seg model is due to the best ratio of model accuracy to execution time at present. Additionally, the accuracy of this model scales well with increasing number of parameters.

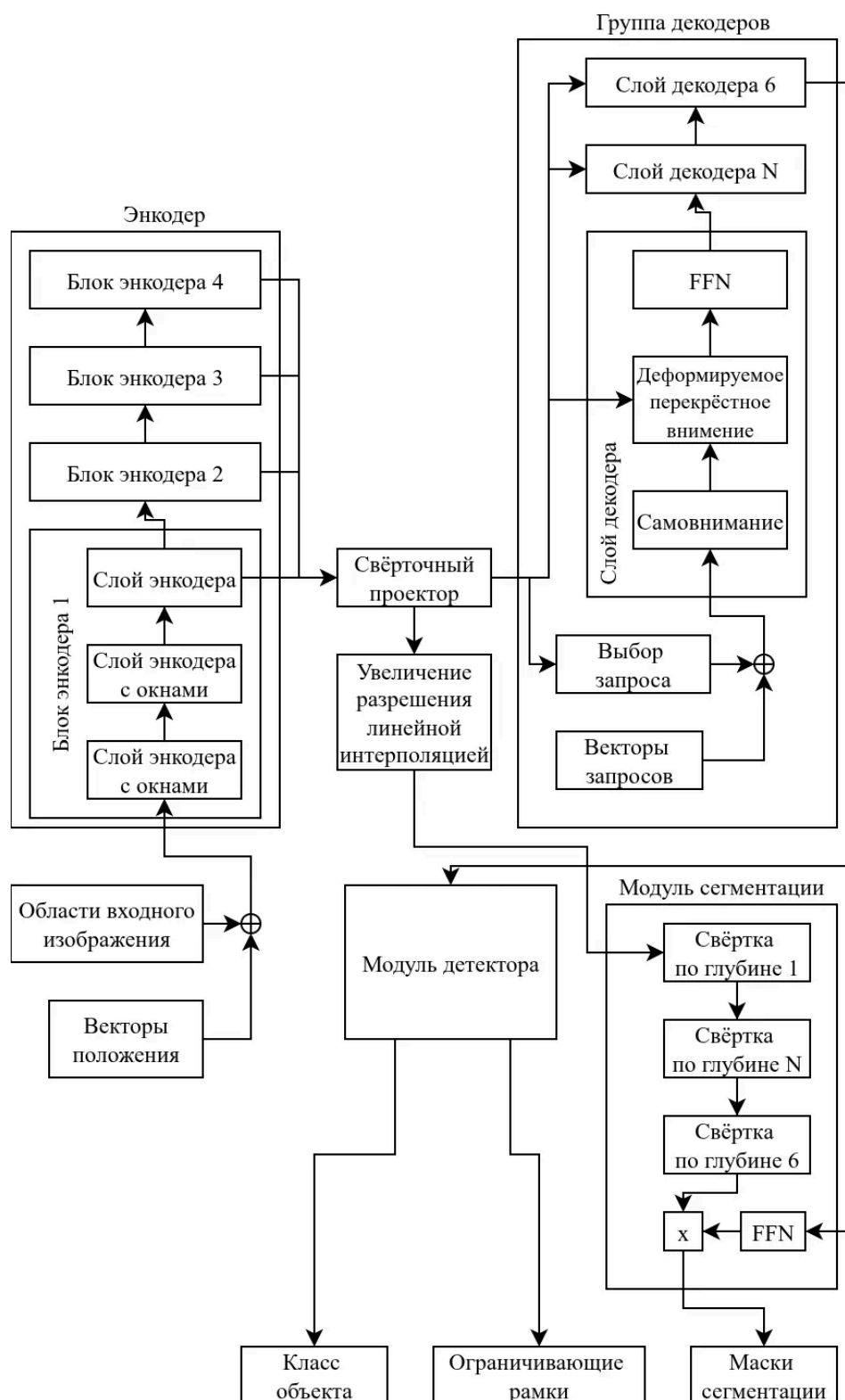


Figure 9 – RF-DETR-seg model architecture diagram

1.5 Object Detection and Tracking

An automotive vision system must extract several parameters of objects, one of which is the speed of these objects. Since cameras cannot measure object speeds directly, speed can be computed by knowing the object's position at two points in

time, for which correspondences must be created between instances of detecting the same object at different times (on different frames).

The task of finding correspondences between the current instance of an object detection and past instances for each object in the scene, usually with assigning a unique identifier to each object, is called the multi-object tracking task [13].

Two main paradigms of object tracking are distinguished: tracking-by-detection and end-to-end tracking [13].

In the tracking-by-detection paradigm, tracking is performed in two steps: first, a detector model detects bounding boxes of objects on each frame; second, objects are associated between frames. This paradigm is used quite often due to modularity: it is possible to use any model as a detector, which is relevant in this work to avoid unnecessary inference costs, and it is also possible to integrate both classical algorithms and machine learning models into the association module. Moreover, this approach achieves high accuracy and performance in practice [13].

The association module is in turn divided into the following parts: motion model, association method, track management logic (an object track contains information related to a specific tracked object, including its identifier).

The motion model predicts the positions of objects on subsequent frames. For prediction, both classical methods based on Kalman filter and machine learning-based methods can be used.

Let z_i be the current state of an object at time t_i , F – the state transition matrix (i.e., the motion model). Then the Kalman filter, which allows computing the mean $\hat{z}_{i+1}$ and covariance $\hat{P}_{i+1}$, is described as follows:

$$\hat{z}_{i+1} = F\tilde{z}_i$$

$$\hat{P}_{i+1} = F\tilde{P}_iF^T + Q$$

Upon receiving a new observation x , the distribution parameters $N(\tilde{z}_{i+1}, \tilde{P}_{i+1})$ are updated as follows:

$$\hat{x}_{i+1} = H\hat{z}_{i+1}$$

$$\hat{\Sigma}_{i+1} = H\hat{P}_{i+1}H^T + R$$

$$K_{i+1} = \hat{P}_{i+1}H^T\hat{\Sigma}_{i+1}^{-1}$$

$$\Delta x_{i+1} = x_{i+1} - \hat{x}_{i+1}$$

$$\tilde{z}_{i+1} = \hat{z}_{i+1} + K_{i+1} \Delta x_{i+1}$$

$$\tilde{P}_{i+1} = \hat{P}_{i+1} - K_{i+1} \hat{\Sigma}_{i+1} K_{i+1}^T$$

Here H is a matrix that transforms the state z into the observation x . The matrix K is called the Kalman gain. Σ is an intermediate covariance matrix. Matrices Q and R represent process noise and measurement noise, respectively, and are computed using various heuristics.

In SORT [14], the state z and measurement x for each object are described as

$$z = [u \ v \ s \ r \ \dot{u} \ \dot{v} \ \dot{s}]^T,$$

$$x = [u \ v \ s \ r]^T,$$

where u, v are pixel coordinates of the object's bounding box, s is the area of the box, r is the aspect ratio, $\dot{u}, \dot{v}$ are the object's speed in pixels per iteration, $\dot{s}$ is the rate of change of the bounding box area. The aspect ratio is assumed to be constant. A linear Kalman filter with the following F and H matrices is used for state update:

$$F = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

When substituted into $\hat{z}_{i+1} = F\tilde{z}_i$, such an F matrix is equivalent to the expressions

$$u_{i+1} = u_i + \dot{u}_i,$$

$$v_{i+1} = v_i + \dot{v}_i,$$

$$s_{i+1} = s_i + \dot{s}_i,$$

$$r_{i+1} = r_i,$$

$$\dot{u}_{i+1} = \dot{u}_i,$$

$$\dot{v}_{i+1} = \dot{v}_i,$$

$$\dot{s}_{i+1} = \dot{s}_i$$

In MotionTrack [15], a transformer-architecture model is proposed for predicting motion, taking into account both the object’s position on past frames and its interaction with other objects that affects the trajectory.

The association method is responsible for defining the association cost function, after which the Hungarian algorithm is used to solve the assignment problem by minimizing total cost. The cost function can be defined as the negative IoU of the new bounding box and the box predicted by the motion model. Additionally, extra terms can be added to the cost function: DeepSort [16], StrongSORT [17], BoT-SORT [18] and others use a re-identification model to extract appearance features of objects and account for the distance between features in the cost function; SparseTrack [19] uses an approximate estimate of the distance to the object (due to perspective, the farther the object, the higher the center of the bounding box on the image).

There are also modifications of the association algorithm: ByteTrack [20] makes the assumption that low-confidence detection instances can be used, and divides all detections into high-confidence and low-confidence detections. In the first step, association of high-confidence detections is performed, after which association of tracked objects for which no association has been found yet and low-confidence detections is performed. ImprAsso [21] proposes a further improvement to the association process, in which association is performed in one step but with the ability to use different cost evaluation functions (in particular, appearance features cannot be used for low-confidence detections) for low-confidence and high-confidence detections, while the cost function for low-confidence detections is multiplied by a factor $\beta = \frac{d_{\max}^h}{d_{\max}^l}$, where $d_{\max}^h$ and $d_{\max}^l$ are the maximum values of the cost function among all high-confidence and low-confidence detections, respectively.

Track management logic defines possible track states and the algorithm for transitioning between them. In SORT [14], there are 4 track states: active (for objects that have a successful association with a detection in the previous frame), lost (for objects that currently have no successful association. Lost tracks transition to the active state when an association appears, or to the deleted state if an association cannot be found for T_{lost} frames), deleted (such objects are no longer tracked),

new (tracks start in this state and transition to “active” after several consecutive successful associations, otherwise the track is deleted).

While not useful for the task at hand, existence of offline methods should be noted. StrongSORT++ [17], MPNTrack [22] achieve higher accuracy but cannot operate in real time, since such methods require all object detections to be available in advance.

In the end-to-end paradigm, both detection and association are performed by a single model. An example of a model following this paradigm is MOTR – an extension of the DETR architecture that adds track queries to object queries.

The end-to-end paradigm has the following disadvantages compared to tracking-by-detection [13]:

- Requires large amounts of resources and data for training.
- Requires a dataset with labeled tracks for training.
- Slower during inference.
- Performs worse with a large number of objects in the scene.

Based on this, the object tracker used in this work will follow the tracking-by-detection paradigm, using the detector model from section 1.4, a Kalman filter-based motion model, and an association algorithm based on ImprAsso with depth consideration, similar to SparseTrack.

1.5.1 Methods and Metrics for Distance Estimation to Objects

There are several metrics for evaluating depth estimation models, among which one can highlight δ_1 (also written as D1), AbsRel, and RMSE.

In the KITTI 2015 benchmark for stereo depth estimation, the primary metric is δ_1 , which describes the percentage of pixels whose estimated and true depth are within 25% of each other. Formally, δ_1 is the percentage of pixels for which the inequality $\max\left(\frac{d_i}{D_i}, \frac{D_i}{d_i}\right) < 1.25^1$ holds, where D_i is the true depth of pixel number i , and d_i is the estimated depth of the pixel. By analogy, metrics δ_2 and δ_3 use values 1.25^2 and 1.25^3 respectively as thresholds.

The AbsRel metric describes the relative mean absolute error and is written as [23]

$$\text{AbsRel}(D, d) = \frac{1}{N} \sum_{i=1}^N \frac{|D_i - d_i|}{d_i}$$

The RMSE metric describes the relative root mean square error and is written as [23]

$$\text{RMSE}(D, d) = \frac{1}{N} \sqrt{\sum_{i=1}^N (D_i - d_i)^2}$$

In the work [24], it is assumed that existing metrics do not fully describe the accuracy of depth estimation algorithms in autonomous driving scenarios, which is why the work introduces metrics ARD and MR, describing different levels of error depending on distance and semantic class of the object, respectively.

ARD_k is a modified version of the AbsRel metric, where only pixels whose depth falls within range k are considered during computation. MR_k is a modified version of the δ_1 metric, where only pixels belonging to class k are considered during computation.

References

- [1] Z. Dai, A. Wolf, P.-P. Ley, T. Glück, M. C. Sundermeier, and R. Lachmayer, “Requirements for Automotive LiDAR Systems,” *Sensors (Basel)*, vol. 22, no. 19, Oct. 2022.
- [2] R. Sun *et al.*, “A millimeter-wave automotive radar with high angular resolution for identification of closely spaced on-road obstacles,” *Scientific Reports*, vol. 13, no. 1, p. 3233, Feb. 2023, doi: 10.1038/s41598-023-30406-4.
- [3] X. Wang, M. A. Maleki, M. W. Azhar, and P. Trancoso, “Moving Forward: A Review of Autonomous Driving Software and Hardware Systems.” [Online]. Available: <https://arxiv.org/abs/2411.10291>
- [4] E. Talpes *et al.*, “Compute Solution for Tesla's Full Self-Driving Computer,” *IEEE Micro*, vol. 40, no. 2, pp. 25–35, 2020, doi: 10.1109/MM.2020.2975764.
- [5] T. Schöps, V. Larsson, M. Pollefeys, and T. Sattler, “Why Having 10,000 Parameters in Your Camera Model is Better Than Twelve.” [Online]. Available: <https://arxiv.org/abs/1912.02908>
- [6] [Online]. Available: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

- [7] H. Hirschmuller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, doi: 10.1109/TPAMI.2007.1166.
- [8] J.-R. Chang and Y.-S. Chen, “Pyramid Stereo Matching Network,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 5410–5418.
- [9] G. Xu *et al.*, “BANet: Bilateral Aggregation Network for Mobile Stereo Matching,” *arXiv preprint arXiv:2503.03259*, 2025.
- [10] G. Csurka, R. Volpi, and B. Chidlovskii, “Semantic Image Segmentation: Two Decades of Research.” [Online]. Available: <https://arxiv.org/abs/2302.06378>
- [11] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, Nov. 2023, doi: 10.3390/make5040083.
- [12] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, “RF-DETR: Neural Architecture Search for Real-Time Detection Transformers.” [Online]. Available: <https://arxiv.org/abs/2511.09554>
- [13] M. Adžemović, “Deep Learning-Based Multi-Object Tracking: A Comprehensive Survey from Foundations to State-of-the-Art.” [Online]. Available: <https://arxiv.org/abs/2506.13457>
- [14] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and real-time tracking,” in *2016 IEEE International Conference on Image Processing (ICIP)*, IEEE, Sept. 2016, pp. 3464–3468. doi: 10.1109/icip.2016.7533003.
- [15] Z. Qin, S. Zhou, L. Wang, J. Duan, G. Hua, and W. Tang, “MotionTrack: Learning Robust Short-term and Long-term Motions for Multi-Object Tracking.” [Online]. Available: <https://arxiv.org/abs/2303.10404>
- [16] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric.” [Online]. Available: <https://arxiv.org/abs/1703.07402>
- [17] Y. Du *et al.*, “StrongSORT: Make DeepSORT Great Again.” [Online]. Available: <https://arxiv.org/abs/2202.13514>

- [18] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust Associations Multi-Pedestrian Tracking.” [Online]. Available: <https://arxiv.org/abs/2206.14651>
- [19] Z. Liu, X. Wang, C. Wang, W. Liu, and X. Bai, “SparseTrack: Multi-Object Tracking by Performing Scene Decomposition based on Pseudo-Depth.” [Online]. Available: <https://arxiv.org/abs/2306.05238>
- [20] Y. Zhang *et al.*, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box.” [Online]. Available: <https://arxiv.org/abs/2110.06864>
- [21] D. Stadler and J. Beyerer, “An Improved Association Pipeline for Multi-Person Tracking,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, pp. 3170–3179. doi: 10.1109/CVPRW59228.2023.00319.
- [22] G. Brasó and L. Leal-Taixé, “Learning a Neural Solver for Multiple Object Tracking.” [Online]. Available: <https://arxiv.org/abs/1912.07515>
- [23] H. Li, C. Wang, J. Lei, K. Daniilidis, and L. Liu, “StereoDiff: Stereo-Diffusion Synergy for Video Depth Estimation.” [Online]. Available: <https://arxiv.org/abs/2506.20756>
- [24] G. Yang, X. Song, C. Huang, Z. Deng, J. Shi, and B. Zhou, “DrivingStereo: A Large-Scale Dataset for Stereo Matching in Autonomous Driving Scenarios,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

Glossary

#	English	Russian
1	unmanned vehicles (UVs)	беспилотные транспортные средства (БТС)
2	autopilot	автопилот
3	classical approach	классический подход
4	modular approach	модульный подход
5	end-to-end approach	end-to-end подход / подход “от сенсора к актуатору”
6	mapping	картографирование
7	sensors	сенсоры / датчики
8	perception	восприятие
9	localization	локализация
10	trajectory planning	планирование траектории
11	prediction of trajectories	предсказание траекторий
12	control (system)	управление (система)
13	actuators	исполнительные устройства / приводы
14	simulation subsystem	подсистема симуляции
15	cameras	камеры
16	lidars	лидары
17	radars	радары
18	GPS	GPS / глобальная навигационная спутниковая система
19	IMU (Inertial Measurement Unit)	ИНУ (инерциальное измерительное устройство)
20	odometer	одометр
21	cost volume	оценочный объём / cost volume
22	disparity map	карта несоответствий / карта диспаратности
23	point cloud	облако точек
24	pinhole camera model	модель камеры с точечной диафрагмой
25	focal length	фокусное расстояние
26	principal point	главный пункт / оптический центр

27	radial distortion	радиальное искажение
28	tangential distortion	тангенциальное искажение
29	thin prism model	модель тонкой призмы
30	parametric camera model	параметрическая модель камеры
31	generic camera model	общая (generic) модель камеры
32	calibration pattern	калибровочный узор / паттерн
33	chessboard pattern	шахматная доска
34	AprilTag marker	AprilTag метка
35	intrinsic parameters	внутренние параметры (интринсики)
36	extrinsic parameters	внешние параметры (экстринсики)
37	rotation matrix	матрица вращения / матрица поворота
38	offset vector	вектор смещения
39	rectification	выравнивание / ректификация
40	disparity	несоответствие / диспаратность
41	parallax effect	эффект параллакса
42	perspective matrix	матрица перспективы
43	depth estimation	оценка глубины
44	stereo pair	стереопара
45	block matching	блочное соответствие / block matching
46	semiglobal block matching	полуглобальное блочное соответствие
47	deep learning	глубокое обучение
48	convolutional neural network (CNN)	свёрточная нейронная сеть (CHC)
49	feature extraction	извлечение признаков / feature extraction
50	spatial pyramid pooling	пирамидальный пулинг / Spatial Pyramid Pooling
51	3D convolution	3D свёртка
52	regression	регрессия
53	softmax	softmax
54	bilateral aggregation	билатеральная агрегация

55	semantic segmentation	семантическая сегментация
56	instance segmentation	сегментация экземпляров
57	bounding box	ограничивающая рамка / bounding box
58	YOLOv8-seg	YOLOv8-seg
59	RF-DETR-seg	RF-DETR-seg
60	C2f layer	слой C2f
61	Cross Stage Partial Bottleneck	межстадийное частичное узкое место (CSP Bottleneck)
62	transformer architecture	архитектура трансформер
63	ViT backbone	ViT-бэкбоунд / основа Vision Transformer
64	DINOv2	DINOv2
65	convolutional projector	свёрточный проектор
66	decoder layers	слои декодера
67	query tokens	токены-запросы / query tokens
68	detection head	модуль детектора / detection head
69	segmentation head	модуль сегментации / segmentation head
70	Neural Architecture Search (NAS)	поиск архитектуры нейронной сети (NAS)
71	COCO dataset	датасет COCO
72	AP (Average Precision)	AP (средняя точность)
73	AP50	AP50
74	inference time	время выполнения / время инференса
75	multi-object tracking	отслеживание множества объектов / MOT
76	tracking-by-detection	отслеживание по обнаружениям
77	Kalman filter	фильтр Калмана
78	state transition matrix	матрица перехода между состояниями
79	covariance	ковариация
80	Kalman gain	коэффициент усиления Калмана
81	process noise	шум процесса

82	measurement noise	шум измерений
83	SORT algorithm	алгоритм SORT
84	DeepSORT	DeepSORT
85	StrongSORT	StrongSORT
86	BoT-SORT	BoT-SORT
87	ByteTrack	ByteTrack
88	ImprAsso	ImprAsso
89	SparseTrack	SparseTrack
90	MOTR	MOTR
91	DETR (DEtection TRansformer)	DETR
92	track association	ассоциация треков
93	Hungarian algorithm	Венгерский алгоритм
94	cost function	функция стоимости / функция потерь
95	IoU (Intersection over Union)	IoU (пересечение над объединением)
96	re-identification model	модель повторной идентификации / ReID
97	appearance features	признаки внешнего вида / дескрипторы внешности
98	aspect ratio	соотношение сторон / aspect ratio
99	track states	состояния трека
100	active track	активный трек
101	lost track	потерянный трек
102	deleted track	удалённый трек
103	new track	новый трек
104	offline tracking	оффлайн-трекинг
105	real-time processing	обработка в реальном времени
106	frame rate	частота кадров
107	GPU (Graphics Processing Unit)	GPU / графический процессор
108	NNA (Neural Network Accelerator)	нейроускоритель (NNA)

109	ISP (Image Signal Processor)	сигнальный процессор для изображений (ISP)
110	FPGA (Field-Programmable Gate Array)	массив программируемых логических вентилей (FPGA)
111	system-on-chip (SoC)	система-на-чипе (SoC)
112	FSD chip	чип FSD
113	Apollo platform	платформа Apollo
114	CenterPoint model	модель CenterPoint
115	darkSCNN	darkSCNN
116	Faster-RCNN	Faster-RCNN
117	RegNet	RegNet
118	bidirectional feature pyramid	двунаправленная пирамида признаков
119	spatial queue	пространственная очередь
120	temporal queue	временная очередь
121	spatial recurrent neural network	пространственная рекуррентная нейронная сеть
122	point cloud processing	обработка облаков точек
123	lane marking detection	обнаружение полос / разметки дороги
124	traffic light detection	обнаружение светофоров
125	object detection	обнаружение объектов
126	pixel coordinates	пиксельные координаты
127	world coordinate system	система координат мира
128	camera distortion	искажение камеры / дисторсия
129	lens distortion compensation	компенсация искажения линз
130	control points	управляющие точки
131	cubic interpolation	кубическая интерполяция
132	sigmoid function	сигмоидальная функция / сигмоид
133	concatenation operation	операция конкатенации
134	upsampling	апсемплинг / увеличение разрешения
135	activation function	функция активации
136	weight sharing	разделение весов

137	transfer learning	трансферное обучение / перенос обучения
138	training dataset	обучающий датасет
139	benchmark	бенчмарк
140	delta_1 metric	метрика delta_1 (D1)
141	AbsRel metric	метрика AbsRel
142	RMSE metric	метрика RMSE
143	ARD metric	метрика ARD
144	MR metric	метрика MR
145	true depth	истинная глубина
146	estimated depth	оценённая глубина
147	relative mean absolute error	относительная средняя абсолютная ошибка
148	root mean square error	среднеквадратичная ошибка (RMSE)
149	KITTI benchmark	бенчмарк KITTI
150	semantic class	семантический класс
151	motion model	модель движения
152	observation / detection	наблюдение / обнаружение
153	state estimation	оценка состояния
154	measurement update	обновление по измерению
155	prediction step	шаг предсказания
156	update step	шаг обновления
157	confidence score	уверенность / скор уверенности
158	high-confidence detection	обнаружение с высокой уверенностью
159	low-confidence detection	обнаружение с низкой уверенностью
160	motion tracking	отслеживание движения / Motion-Track
161	interaction modeling	моделирование взаимодействия
162	perspective distance estimation	оценка расстояния по перспективе
163	illumination features	признаки освещённости
164	texture features	признаки текстуры

165	acceleration	ускорение
166	velocity / speed	скорость
167	trajectory prediction	предсказание траектории
168	actuator signals	сигналы исполнительных устройств
169	feedback loop	контур обратной связи
170	global trajectory	глобальная траектория
171	local trajectory	локальная траектория
172	navigation	навигация
173	obstacle detection	обнаружение препятствий
174	static obstacles	статические препятствия
175	moving objects	движущиеся объекты
176	road geometry	геометрия дороги
177	occluded objects	закрытые (окклюзированные) объекты
178	feature pyramid	пирамида признаков
179	residual connection	остаточная связь / skip connection
180	encoder-decoder architecture	архитектура энкодер-декодер
181	attention mechanism	механизм внимания
182	self-attention	самовнимание
183	multi-head attention	многоголовое внимание / multi-head attention
184	positional encoding	позиционное кодирование
185	token	токен
186	embedding	эмбединг / векторное представление
187	batch size	размер батча
188	learning rate	скорость обучения / learning rate
189	loss function	функция потерь
190	optimization algorithm	алгоритм оптимизации
191	gradient descent	градиентный спуск
192	backpropagation	обратное распространение ошибки
193	overfitting	переобучение
194	underfitting	недообучение
195	regularization	регуляризация

196	dropout	dropout / отключение нейронов
197	data augmentation	аугментация данных
198	inference	инференс / вывод
199	latency	задержка / латентность
200	throughput	пропускная способность
201	power consumption	энергопотребление
202	hardware accelerator	аппаратный ускоритель
203	direct memory access	прямой доступ к памяти / DMA
204	sensor fusion	слияние данных с датчиков / sensor fusion
205	multi-sensor system	мультисенсорная система
206	camera calibration	калибровка камеры
207	stereo calibration	стереокалибровка
208	distortion model	модель искажений
209	mapping parameters	параметры отображения
210	rectified image	выровненное изображение
211	source image	исходное изображение
212	correspondence map	карта соответствий
213	width dimension	измерение ширины
214	height dimension	измерение высоты
215	channel dimension	измерение каналов
216	tensor	тензор
217	kernel size	размер ядра
218	stride	шаг свёртки / stride
219	padding	заполнение / padding
220	pooling layer	слой пулинга
221	max pooling	max pooling / максимальный пулинг
222	average pooling	average pooling / средний пулинг
223	feature map	карта признаков
224	receptive field	рецептивное поле
225	dilation	дилатация / dilated convolution
226	skip connection	skip-связь / связь с пропуском
227	residual block	остаточный блок / residual block
228	bottleneck structure	структура «бутылочного горлышка»

229	depthwise convolution	поканальная свёртка / depthwise convolution
230	pointwise convolution	точечная свёртка / pointwise convolution
231	group convolution	групповая свёртка / group convolution
232	normalization layer	слой нормализации
233	batch normalization	пакетная нормализация / batch normalization
234	layer normalization	послойная нормализация / layer normalization
235	ReLU activation	функция активации ReLU
236	GELU activation	функция активации GELU
237	cross-entropy loss	функция потерь cross-entropy
238	focal loss	фокусная функция потерь / focal loss
239	smooth L1 loss	гладкая L1-функция потерь / smooth L1 loss
240	classification head	модуль классификации / classification head
241	bounding box regression	регрессия ограничивающей рамки
242	mask prediction	предсказание маски
243	pixel-wise prediction	попиксельное предсказание
244	objectness score	оценка наличия объекта / objectness score
245	class label	метка класса / class label
246	false positive	ложноположительный результат
247	false negative	ложноотрицательный результат
248	precision	точность (precision)
249	recall	полнота (recall)
250	F1 score	F1-мера
251	mAP (mean Average Precision)	mAP (средняя средняя точность)
252	dataset annotation	аннотация датасета
253	ground truth	эталонные данные / ground truth
254	validation set	валидационная выборка

255	test set	тестовая выборка
256	cross-validation	кросс-валидация
257	hyperparameter tuning	настройка гиперпараметров
258	grid search	поиск по сетке / grid search
259	stochastic gradient descent (SGD)	стохастический градиентный спуск (SGD)
260	Adam optimizer	оптимизатор Adam
261	weight decay	затухание весов / weight decay
262	momentum	импульс / momentum
263	learning rate scheduler	планировщик скорости обучения
264	warmup phase	фаза разогрева / warmup
265	cosine annealing	косинусное затухание / cosine annealing
266	early stopping	ранняя остановка
267	model checkpoint	контрольная точка модели
268	pre-trained model	предобученная модель
269	fine-tuning	тонкая настройка / fine-tuning
270	zero-shot inference	инференс в формате zero-shot
271	few-shot learning	обучение с малым количеством примеров / few-shot learning
272	domain adaptation	адаптация к домену
273	synthetic data	синтетические данные
274	simulation environment	среда симуляции
275	real-world data	реальные данные / данные из реального мира
276	edge computing	граничные вычисления / edge computing
277	embedded system	встроенная система
278	real-time operating system (RTOS)	операционная система реального времени (RTOS)
279	ROS (Robot Operating System)	ROS (операционная система роботов)
280	OpenCV	OpenCV
281	PyTorch	PyTorch
282	TensorFlow	TensorFlow

283	ONNX format	формат ONNX
284	model quantization	квантование модели
285	model pruning	прунинг / обрезка модели
286	knowledge distillation	дистилляция знаний
287	inference engine	движок инференса
288	tensor core	тензорное ядро
289	compute throughput	вычислительная пропускная способность
290	memory bandwidth	пропускная способность памяти
291	power budget	бюджет энергопотребления
292	thermal throttling	тепловое троттлинг / ограничение по температуре
293	sensor noise	шум датчика
294	weather effects	погодные условия / влияние погоды
295	lighting conditions	условия освещения
296	dynamic range	динамический диапазон
297	field of view (FOV)	поле зрения (FOV)
298	angular resolution	угловое разрешение
299	distance resolution	разрешение по расстоянию
300	latency budget	бюджет задержки / латентности